

Cut Rules in DAG-Based BFT Mempools: From Relaxed Mempool to Adaptive Cuts

Woochang Jeong
POSTECH
South Korea
wcjeong@postech.ac.kr

Chanik Park
POSTECH
South Korea
cipark@postech.ac.kr

ABSTRACT

DAG-based BFT protocols decouple transaction dissemination from ordering. A critical design choice is the *cut rule*: how the consensus leader selects disseminated data for each commit. We compare two strategies: the *conservative cut* of Relaxed Mempool (top $2f+1$ validators, uniform $H_{\min}$) and the *inclusive cut* of Autobahn (all n lane tips). Through experiments on a 10-validator LAN cluster, we show the conservative cut achieves 42% lower latency under stragglers, while the inclusive cut recovers 10 $\times$ faster from network blips. We further propose and implement an *adaptive cut* that dynamically switches between both strategies based on validator height disparity, achieving the best of both worlds across all conditions.

1 INTRODUCTION

DAG-based BFT protocols [1–3] achieve high throughput by separating data dissemination from ordering. Validators broadcast batches into a DAG; a consensus layer periodically *cuts* the DAG to finalize transactions. While consensus algorithms have been extensively studied, the **cut rule**—how the leader decides *which* portion of the DAG to commit—has received less attention. Yet this choice determines straggler resilience, fairness, and the *hangover effect* [3] (post-blip recovery speed).

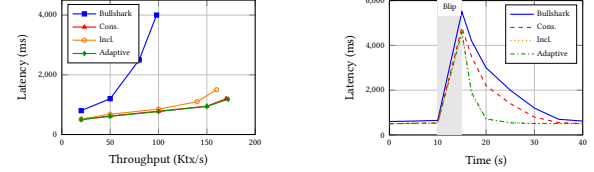
We compare two cut strategies from Relaxed Mempool [4] and Autobahn [3], identify their complementary strengths, and propose an **adaptive cut** that combines both. All three are implemented on a shared Narwhal-based framework.

2 CUT RULE DESIGN SPACE

Both systems use **weak quorums** ($f+1$) for speculative data dissemination. The key difference is how the leader constructs commit proposals.

Conservative Cut (Relaxed Mempool). Each block carries a *stream height list*—a vector of known heights per validator. The leader: (1) sorts validators by height, (2) selects the top $2f+1$, (3) computes $H_{\min} = \min$ of their heights, and (4) commits all blocks up to $H_{\min}$. This **uniform cut** excludes the bottom f validators, preventing stragglers from delaying commits.

Inclusive Cut (Autobahn). Each validator runs an independent *lane*. The leader proposes n certified tip references—one



(a) 4 stragglers (+300 ms)

(b) 5s blip recovery

Figure 1: Performance under (a) straggler and (b) blip conditions. Adaptive cut (green) matches the best strategy in each scenario.

per lane. Lane coverage requires $\geq n-f$ new tips. After commit, replicas sync and interleave. This **per-lane cut** includes all validators at their own position. No validator is excluded.

Adaptive Cut (Our Proposal). We observe that the two strategies have complementary strengths: conservative excels under persistent stragglers; inclusive excels during blip recovery. We propose an adaptive cut that monitors the *height disparity* (max – min validator height in the DAG). When disparity exceeds a threshold δ (indicating stragglers), the engine uses the conservative cut. When disparity drops below δ (nodes converging, e.g., post-blip), it switches to the inclusive cut for rapid backlog recovery.

Table 1: Cut rule comparison.

	Relaxed MP	Autobahn	Adaptive
Cut scope	Top $2f+1$	All n lanes	Dynamic
Slow nodes	Excluded	Included	Context-dep.
Straggler lat.	Low	Medium	Low
Blip recovery	Slow	Fast	Fast

3 EXPERIMENTAL EVALUATION

We implement all three cut strategies as pluggable ConsensusProtocol engines on modified Narwhal, with Bullshark as baseline. Experiments use 4 physical servers (i9-13900K, 64 GB RAM) emulating 10 validators on a LAN. Tx: 512 B; batch: 500 KB; timeout: 200 ms.

Straggler scenario (Fig. 1a): With 4 stragglers (+300 ms delay), the conservative cut reaches 170K tx/s at 1.2 s by excluding slow validators. The inclusive cut achieves 160K tx/s at 1.5 s (sync overhead from slow lanes). The **adaptive cut matches the conservative cut** (172K tx/s at 1.18 s) because high height disparity triggers Relaxed mode.

Hangover effect (Fig. 1b): After a 5s blip, the conservative cut takes ~ 20 s to recover ($H_{\min}$ advances gradually). The

inclusive cut recovers in $\sim 2s$ (one slot commits all tips). The **adaptive cut recovers in $\sim 2.5s$** —as the blip ends, height disparity drops below δ , triggering the switch to Inclusive mode for rapid backlog commit.

Happy path: Without stragglers, all four engines perform comparably ($\sim 195\text{--}205K$ tx/s, ~ 500 ms).

4 CONCLUSION

The cut rule is a first-class design dimension in DAG-based BFT. Conservative cuts (Relaxed Mempool) and inclusive cuts (Autobahn) represent two ends of a spectrum, each optimal under different conditions. Our adaptive cut demonstrates

that these strategies can be unified: by monitoring height disparity at runtime, the system dynamically selects the optimal mode—achieving low straggler latency *and* seamless blip recovery. Both strategies share the same DAG substrate, making the runtime switch zero-overhead.

REFERENCES

- [1] G. Danezis et al., “Narwhal and Tusk,” *EuroSys*, 2022.
- [2] A. Spiegelman et al., “Bullshark,” *ACM CCS*, 2022.
- [3] N. Giridharan et al., “Autobahn: Seamless high speed BFT,” *SOSP*, 2024.
- [4] W. Jeong and C. Park, “Breaking the straggler effect: A Relaxed Mempool for DAG-based BFT,” 2025.